

MONATAC

Documentacion Tecnica del Videojuego

Juego de combate por turnos con dados y cartas — Multijugador local (HotSeat)
2-4 jugadores

Programacion en Videojuegos II — Parcial 1 (Hito 5)

Autor: Braian Zapater

Motor: Unity 6000.3.8f1 · Lenguaje: C#

Repositorio (GitHub)	github.com/wackxion/MONATAC
Diagrama de flujo (Figma)	figma.com/board/MONATAC-flujo
Gestion de tareas (Trello)	Tablero de Trello del proyecto
Build jugable (Drive)	Carpeta de Google Drive

1. Descripción del juego

MONATAC es un juego de mesa digital de **combate por turnos** donde la estrategia, la economía de recursos y la suerte de los dados determinan quien es el ultimo en pie. Es multijugador local (HotSeat) para 2 a 4 jugadores en la misma pantalla. Cada turno el jugador elige una accion (atacar, curarse, recolectar o descartar) y potencia sus jugadas con cartas compradas del mazo.

Campo	Detalle
Genero	Combate por turnos con gestion de recursos
Jugadores	2 a 4 (multijugador local, HotSeat)
Duracion	15-30 minutos
Victoria	Ultimo jugador vivo, o el de mas HP al agotarse las rondas

Mecanicas principales

- **4 acciones** por turno (jerarquia polimorfica Accion): Atacar (3d4), Curarse (2d4), Recolectar (3d4 monedas), Descartar.
- **Dados** de 4 caras (d4) como fuente de azar.
- **Sistema de cartas (54)**: compra (6 monedas), mano de hasta 5, cartas de bonus/comodines/vencimiento, defensivas automaticas, grupales y mazo circular.
- **Seleccion de objetivo** (rota entre rivales vivos) y **configuracion inicial** (jugadores, HP, rondas).

2. Arquitectura de capas

El codigo esta separado en **tres capas** con responsabilidades distintas. Las capas **/Data** y **/Rules** son **C# puro** (no dependen de Unity), lo que permite testearlas sin abrir el juego y protege la logica frente a un cambio de motor.

Capa	Responsabilidad	Clases principales
/Data	Estado y entidades (C# puro)	Jugador, Carta (+subtipos), Mazo, PilaDescarte, Config, Reglas, Enums
/Rules	Logica y reglas (C# puro)	Partida, GestorCartas, Accion (+subtipos), FabricaDeCartas, Dado, ContextoGrupal, IPartida, IGestorCartas, IVistaJuego, PresentadorJuego
/Visual	Presentacion e input (Unity)	GameManager, VistaJuegoUI, GestorAudio, Managers de menus, Animaciones

Escenas: Splash (intro) · MENU · Loading · SeleccionPersonajes · Personalizacion · juego.

3. Estructura de clases (UML)

Relaciones destacadas del diagrama de clases (ver enlace de Figma en la portada):

- **Herencia / polimorfismo:** Carta y Accion son clases **abstractas** con sus subtipos, que redefinen el comportamiento con override.
- **Interfaces (DIP):** Partida implementa IPartida; GestorCartas implementa IGestorCartas; GameManager implementa IVistaJuego.

- **MVP:** PresentadorJuego depende de IPartida e IVistaJuego (abstracciones), no de las clases concretas.
- **Composicion:** Partida contiene la lista de Jugador; cada Jugador contiene su mano (List<Carta>).

4. Patrones de diseno y SOLID

Patron principal: MVP (Modelo-Vista-Presentador) + DIP

Desacopla las **reglas** de la **interfaz de Unity**. La Vista y el Modelo nunca se comunican directo: la Vista dispara un **evento** (AlPedirCambiarObjetivo), el Presentador lo escucha, actualiza el Modelo (Partida) y le pide a la Vista que refresque a traves de la interface IVistaJuego. Aplicado en la interaccion "Cambiar Objetivo".

Otros patrones

Patron	Donde	Para que
Singleton	GameManager, GestorAudio	Una unica instancia global; evita duplicados que pisen el estado
Factory	FabricaDeCartas.CrearMazo()	Centraliza la creacion de las 54 cartas en un solo lugar
Herencia + Polimorfismo	Carta, Accion	Agregar una carta/accion = crear una clase, sin if por tipo
Observer (evento)	AlPedirCambiarObjetivo	La Vista avisa sin saber quien escucha

SOLID

- **SRP:** las reglas se extrajeron del GameManager a Partida y GestorCartas; el dibujado a VistaJuegoUI.
- **OCP:** Carta y Accion estan abiertas a extension pero cerradas a modificacion (se crea una clase que hereda y hace override, sin tocar GestorCartas ni GameManager).
- **DIP:** PresentadorJuego depende de IVistaJuego, y GameManager de IPartida / IGestorCartas.

5. Sistemas del juego (Game Feel)

Interfaz de Usuario (UI)

- Barras de HP (Image.fillAmount) coloreadas segun turno/objetivo, indicador de turno, contador de ronda y de monedas, mano de cartas con la marca [USAR].
- Feedback de estado: los botones se iluminan/apagan segun lo que se puede hacer, las cartas usables se resaltan al elegir una accion y las en uso muestran un marco.

Audio (GestorAudio, Singleton)

- Musica de fondo en loop, continua entre escenas (DontDestroyOnLoad).
- SFX de UI (clic en todos los botones, enganchado automaticamente por escena).
- 8 SFX de gameplay: dados, dano, curacion, comprar, cambiar objetivo, recolectar (mucho/poco) y victoria.

Animaciones / VFX

- Dados girando, barra de HP suave (Mathf.Lerp), numeros flotantes de dano/cura, carta grupal con fade, pantalla de carga con barra de progreso.

Flujo y menus (Hito 5)

- Splash con video en loop y boton Jugar.
- Menu de pausa (Esc / boton, congela con Time.timeScale).
- Panel de instrucciones (reglas por fases del turno).
- Pantalla de fin con estadisticas de todos los jugadores.
- Boton Salir (Application.Quit).

6. Flujo del programa

```
SPLASH --> MENU --> LOADING --> SELECCION PERSONAJES --> JUEGO --> FIN
      |                               |
      (Instrucciones / Personalizacion / Salir)       (Pausa con Esc)
```

Arranque (GameManager.Start):

```
lee Config -> new Partida -> Mazo (FabricaDeCartas) -> GestorCartas
               -> PresentadorJuego -> IniciarTurno()
```

Un turno (banderas haElegido / yaTiro / puedeComprar):

1. Elegir accion -> crea una Accion
2. (opcional) Elegir cartas
3. (si atacas) Cambiar Objetivo -> evento -> PresentadorJuego (MVP)
4. Lanzar Dados -> GestorCartas (dados/mult/bonus) -> Dado.Tirar()

-> accion.Aplicar(partida, total)
5. (si recolectaste) Comprar carta
6. Pasar Turno -> Partida.PasarTurno() -> vuelve a 1

Cada transicion -> ActualizarUI() -> VistaJuegoUI dibuja e ilumina.

7. Pruebas unitarias (Testing)

El proyecto incluye **18 tests EditMode / NUnit** (26 casos, varios data-driven con [TestCase]) sobre la logica de dominio, todos en verde. Se corren con Window > General > Test Runner > EditMode > Run All. Se pueden testear porque /Data y /Rules no dependen de Unity.

Archivo	Que prueba
JugadorTests	Dano nunca baja de 0, cura no supera el maximo, gasto de monedas, EstaVivo, monedas negativas
PartidaTests	Victoria por ultimo vivo, pasar turno saltea muertos, fin por rondas, Ley Marcial, cambiar objetivo
CartasTests	AplicaA por tipo de carta, comodines, defensivas, carta de reaccion (absorcion)
MazoTests	Robar vacia el mazo, reciclar recupera el descarte

8. Tecnologias y librerias

Herramienta / Paquete	Version	Uso
Unity	6000.3.8f1	Motor del videojuego

C#	-	Lenguaje de scripting
Input System	1.18.0	Entrada (tecla Esc de pausa, saltar splash)
Universal Render Pipeline (URP)	17.3.0	Render 2D
uGUI + TextMeshPro	2.0.0	Interfaz y textos
Test Framework	1.6.0	Pruebas unitarias (NUnit, EditMode)
Video module	1.0.0	Reproduccion del splash
Git + GitHub	-	Control de versiones

9. Mapa de controles

El juego es 100% con el mouse (HotSeat), con un atajo de teclado para la pausa.

Control	Accion
Clic izquierdo	Todo: elegir accion, marcar cartas [USAR], botones de menu
Botones de accion	Atacar / Curarse / Recolectar / Descartar
Cambiar Objetivo	Rotar el rival apuntado (solo al atacar)
Lanzar Dados	Resolver la jugada
Comprar / Pasar Turno	Comprar carta (6 monedas) / terminar el turno
Tecla Esc (o boton)	Pausar / Reanudar (en gameplay)
Instrucciones / Salir	Abrir el panel de reglas / cerrar el juego (en el menu)

10. Equipo y gestion

Integrante	Rol
Braian Zapater	Programador principal: arquitectura (3 capas, MVP, DIP, patrones), logica de dominio, audio, testing y flujo del Hito 5
Julian Gabriel Blanco	Animaciones/VFX (datos, hover de cartas) y pantalla de personalizacion
Alvarez Pilar	Configuracion de partida, menu de inicio y parte de la logica de Partida

La gestion de tareas por area e hito se lleva en el **Trello** y el archivo Backlog.md del repositorio (ver enlaces en la portada).